

# Day 4 — OOP 深入 & 类型系统（完整版）

## 一、属性控制与内存优化

### 1. 是什么

Python 的属性控制远比" `self.xxx = xxx` "丰富。你可以在不改变外部接口的前提下，控制属性的读写行为、校验逻辑，甚至优化内存占用。

今天要学的：

| 工具                           | 一句话                               | 解决什么问题             |
|------------------------------|-----------------------------------|--------------------|
| <code>property</code>        | 用方法伪装成属性，读写时自动触发逻辑                | 不想破坏已有代码，又需要加校验/计算 |
| <code>__slots__</code>       | 固定实例的属性名，去掉 <code>__dict__</code> | 百万级实例时内存爆了         |
| <code>cached_property</code> | 算一次，缓存后不再重算                       | 昂贵的计算结果反复访问        |

### 2. 解决了什么问题

问题 1：想要校验，但不想破坏外部接口

```
# 用户类，age 不能是负数
class User:
    def __init__(self, age):
        self.age = age # 直接赋值，负数也放行

u = User(-5) # ✕ 没人提醒
```

```

# 传统方案：改成 getter/setter 方法
class User:
    def set_age(self, value):
        if value < 0:
            raise ValueError("年龄不能为负")
        self._age = value
    def get_age(self):
        return self._age

u = User()
u.set_age(-5)  # 可以校验了，但 u.age 不能用了

# 用 property：既保留 u.age 语法，又能校验
class User:
    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):
        if value < 0:
            raise ValueError("年龄不能为负")
        self._age = value

```

## 问题 2：计算属性每次都重算

```

class DataReport:
    def __init__(self, data):
        self.data = data

    @property
    def summary(self):
        print("正在计算...", end=" ")
        return sorted(self.data)[:5]  # 昂贵的排序

r = DataReport([3, 1, 4, 1, 5, 9, 2, 6, 5])
print(r.summary)  # 正在计算...
print(r.summary)  # 正在计算... 每次都重算！

```

用 `@cached_property` 只算一次。

问题 3：百万个实例，内存爆了

```
# 一个 Person 实例有 __dict__, 约 56+ 字节
# 100万个人物 → 56MB
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

# 用 __slots__ 固定属性 → 32 字节, 省 43%
class Person:
    __slots__ = ("name", "age")
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

### 3. 核心理论

#### property

一句话：用方法伪装成属性，读写触发自定义逻辑。

```
class Temperature:
    def __init__(self, celsius=0):
        self._celsius = celsius

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("低于绝对零度!")
        self._celsius = value

    @property
```

```

    def fahrenheit(self):
        return self._celsius * 9/5 + 32

    @fahrenheit.setter
    def fahrenheit(self, value):
        self._celsius = (value - 32) * 5/9

t = Temperature(100)
print(t.fahrenheit)      # 212.0
t.fahrenheit = 32
print(t.celsius)         # 0.0

```

## 应用场景

- 校验：赋值时做类型/范围检查
- 计算属性：属性值来自其他字段计算
- 向后兼容：已有的 `obj.field` 改成方法但不改调用方
- 只读属性：只定义 `@property` 不定义 `setter`

注意：property 读写的性能比直接访问 `self._xxx` 略慢（方法调用开销）。热路径上谨慎使用。

## cached\_property

```

from functools import cached_property

class DataReport:
    def __init__(self, data):
        self.data = data

    @cached_property
    def summary(self):
        print("正在计算...")
        return sorted(self.data)[:5]

r = DataReport([3, 1, 4, 1, 5, 9, 2, 6])
print(r.summary)  # 正在计算... [1, 1, 2, 3, 4]
print(r.summary)  # 直接返回缓存，不重算

```

什么时候用 - 计算结果不变（数据不修改） - 计算开销大 - 同一实例多次访问

什么时候不该用 - 数据会变化（缓存不会自动失效） - 内存敏感（计算结果会一直留着）

`__slots__`

一句话：固定属性名，去掉 `__dict__`，省内存 + 提速度。

```
class Point:
    __slots__ = ("x", "y")

    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Point(1, 2)
print(p.x)          # 1
# p.z = 3            # ✗ AttributeError
# print(p.__dict__)  # ✗ 没有 __dict__

# 内存对比
import sys

class RegularPoint:
    def __init__(self, x, y):
        self.x = x
        self.y = y

r = RegularPoint(1, 2)
s = Point(1, 2)
print(sys.getsizeof(r))  # 56 bytes
print(sys.getsizeof(s))  # 32 bytes - 省 43%
```

有什么代价 - 不能动态添加属性（`a.new_attr = xxx` 会报错） - 不能给实例绑定方法（除非在 `__slots__` 中声明） - 继承时子类也必须定义 `__slots__` 才能利用内存优化 - 多重继承时可能有冲突

应用场景 - 数据类、DTO（数据传输对象）、Entity（领域实体） - 游戏中的坐标/向量/粒子系统（海量实例） - 配置类（属性固定）

## 二、接口与类型系统

### 1. 是什么

当代码规模增大，你需要一种方式来定义"什么对象可以做什么事"。Python 给了你三种选择：

| 工具                             | 哲学          | 检查时机        | 典型场景              |
|--------------------------------|-------------|-------------|-------------------|
| ABC (Abstract Base Class)      | 你继承我，才是我的人  | 实例化时（显式继承）  | 框架、库、需要默认实现       |
| Protocol                       | 你长这样，就是我的类型 | 类型检查时（结构匹配） | 第三方类型适配、鸭子类型+类型安全 |
| <code>__init_subclass__</code> | 有人继承我，我知道   | 子类创建时自动触发   | 插件注册、自动发现         |

### 2. 解决了什么问题

问题 1：如何定义"这个类必须实现 **xxx** 方法"

```
# 没有任何约束，全靠自觉
class Circle:
    def area(self): return 3.14 * r ** 2

class Square:
    def area(self): return s ** 2

# 理想：规定所有 Shape 必须有 area()，漏了报错
```

ABC 解决这个问题：没实现 `area()` 的类根本创建不了实例。

问题 2：如何让第三方类也符合你的接口

```
# 你的框架要求对象有 draw() 方法
def render_all(objects):
    for obj in objects:
        obj.draw()

# 第三方库的图形类有 draw() 但不是你的子类
# → 应该能用，但类型检查会报错（如果有类型检查的话）
# → Protocol 解决：结构匹配，不需要继承
```

### 问题 3：如何自动发现所有插件

```
# 不想手动维护一个插件列表
class AudioPlugin: ...
class VideoPlugin: ...
class ImagePlugin: ...

# plugins = [AudioPlugin, VideoPlugin, ImagePlugin] # □ 每次加插件都要改这行
```

用 `__init_subclass__`：只要有人继承你的基类，自动注册。

## 3. 核心理论

### ABC (Abstract Base Class)

一句话：定义接口契约——子类必须实现哪些方法，少一个都不让你实例化。

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self) -> float: ...

    @abstractmethod
    def perimeter(self) -> float: ...

    def describe(self) -> str:
        return f"面积: {self.area()}, 周长: {self.perimeter()}"
```

```

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self) -> float:
        return 3.14159 * self.radius ** 2

    def perimeter(self) -> float:
        return 2 * 3.14159 * self.radius

# shape = Shape() # ✕ TypeError: Can't instantiate abstract class
circle = Circle(5)
print(circle.describe()) # 面积: 78.53975, 周长: 31.4159

```

## ABC 注册第三方类

不想（或不能）让第三方类继承你的 ABC？注册一下就行：

```

@Shape.register
class ThirdPartyShape:
    def area(self):
        return 42
    def perimeter(self):
        return 0

print(isinstance(ThirdPartyShape(), Shape)) # True

```

## ABC 还能干什么

- `@abstractmethod`、`@abstractclassmethod`、`@abstractproperty`
- 在 `__init_subclass__` 里做自动检查
- 结合 `ABCMeta` 自定义元类行为

什么时候用 **ABC** - 框架/库定义核心接口（Django、Flask、FastAPI 都用） - 需要继承时自动获得默认实现 - 需要 `isinstance` 做运行时类型判断

## Protocol

一句话：鸭子类型 + 类型检查——对象只要能做这件事就行，不需要是一家人。



```

from typing import Protocol

class Drawable(Protocol):
    def draw(self) -> None: ...

class Circle:
    def draw(self) -> None:
        print("画个圆 ◻")

class Square:
    def draw(self) -> None:
        print("画个方 ■")

# Circle 和 Square 不需要继承 Drawable !
def render(obj: Drawable) -> None:
    obj.draw()

render(Circle()) # ✔ 类型检查通过
render(Square()) # ✔
# render(42)      # ✘ mypy 会报错

```

## runtime\_checkable

默认 Protocol 只做静态类型检查。要让 `isinstance` 也能用：

```

from typing import Protocol, runtime_checkable

@runtime_checkable
class Iterable(Protocol):
    def __iter__(self):
        ...

print(isinstance([1, 2], Iterable)) # True
print(isinstance(42, Iterable))    # False

```

但注意：runtime\_checkable 只检查方法存在，不检查签名是否正确。

## ABC vs Protocol 怎么选

| 场景                            | ABC                           | Protocol                            |
|-------------------------------|-------------------------------|-------------------------------------|
| 框架定义接口+需要默认实现                 | ✓ 用 ABC                       | ✗ Protocol 没有默认实现                   |
| 给第三方类加接口                      | ✗ 要改源码                        | ✓ 鸭子类型就行                            |
| 需要 <code>isinstance</code> 检查 | ✓ 默认支持                        | ✓ 加 <code>@runtime_checkable</code> |
| 需要强制继承关系                      | ✓ 显式                          | ✗ 结构匹配                              |
| 运行时性能                         | 略慢                            | 快                                   |
| 典型用途                          | Django Model, FastAPI Depends | 函数参数约束、插件契约                         |

## `__init_subclass__`

一句话：只要有人继承了你的类，自动触发回调。

```
class PluginBase:
    _registry: dict[str, type] = {}

    def __init_subclass__(cls, **kwargs):
        super().__init_subclass__(**kwargs)
        PluginBase._registry[cls.__name__] = cls
        print(f"▣ 新插件注册: {cls.__name__}")

class AudioPlugin(PluginBase): pass
class VideoPlugin(PluginBase): pass
class ImagePlugin(PluginBase): pass

print(PluginBase._registry)
# {'AudioPlugin': ..., 'VideoPlugin': ..., 'ImagePlugin': ...}
```

经典应用：插件系统

```

class PluginBase:
    _registry = {}

    def __init_subclass__(cls, **kwargs):
        super().__init_subclass__(**kwargs)
        cls._order = kwargs.get("order", 100) # 支持排序
        PluginBase._registry[cls.__name__] = cls

    @classmethod
    def run_all(cls):
        """按 order 顺序执行所有插件"""
        for name, plugin_cls in sorted(
            cls._registry.items(),
            key=lambda x: x[1]._order
        ):
            plugin = plugin_cls()
            plugin.execute()

    def execute(self):
        raise NotImplementedError

class LogPlugin(PluginBase, order=1):
    def execute(self):
        print("□ 日志插件运行")

class ValidatePlugin(PluginBase, order=2):
    def execute(self):
        print("✓ 验证插件运行")

PluginBase.run_all()
# □ 日志插件运行
# ✓ 验证插件运行

```

什么时候用 - 插件/扩展注册器 - 自动发现子类（替代入口文件手动 import） - 子类元信息收集（API 路由注册）

---

## 三、数据类与类型注解

### 1. 是什么

写一个"只有数据没有逻辑"的类，在 Python 中有很多多种方式。你写的样板代码越多，维护负担越大。

| 工具                      | 一句话                                                                      | 缺点                                               |
|-------------------------|--------------------------------------------------------------------------|--------------------------------------------------|
| 普通 dict                 | 最灵活，啥都能存                                                                 | 无类型约束，用 <code>['x']</code> 不如 <code>.x</code> 顺手 |
| <code>namedtuple</code> | 不可变的元组 + 字段名                                                             | 不可变、不能加方法                                        |
| <code>dataclass</code>  | 自动生成 <code>__init__</code> 、 <code>__repr__</code> 、 <code>__eq__</code> | 3.7+ 才引入，有性能开销                                   |
| 类型注解                    | 告诉 IDE 和工具这是什么类型                                                         | 运行时不管，但工具链用                                      |

### 2. 解决了什么问题

问题：打个数据类要写一堆样板代码

```
class User:
    def __init__(self, name, email, age=0):
        self.name = name
        self.email = email
        self.age = age

    def __repr__(self):
        return f"User(name={self.name}, email={self.email}, age={self.age})"

    def __eq__(self, other):
        if not isinstance(other, User):
            return NotImplemented
        return (self.name, self.email, self.age) == (other.name, other.email, other.age)
```

```
# 30 行代码，就为了放 3 个字段
```

用 `dataclass`：

```
from dataclasses import dataclass

@dataclass
class User:
    name: str
    email: str
    age: int = 0

# __init__、__repr__、__eq__ 全自动
u = User("Leo", "leo@example.com")
print(u) # User(name='Leo', email='leo@example.com', age=0)
```

### 3. 核心理论

#### `dataclass`

一句话：装饰器自动生成 `__init__`、`__repr__`、`__eq__`、`__hash__`，让你只关注数据字段。

#### 基础用法

```
from dataclasses import dataclass, field, asdict, astuple

@dataclass
class User:
    name: str
    email: str
    age: int = field(default=0, repr=False) # repr 中隐藏
    tags: list[str] = field(default_factory=list) # 不能用 [] 做默认值！

    def __post_init__(self):
        """初始化后自动调用，适合做校验/转换"""
        self.email = self.email.lower()
```

```

u = User("Leo", "Leo@example.com")
print(u.email)      # "leo@example.com"
print(u)            # User(name='Leo', email='leo@example.com')
print(asdict(u))    # {'name': 'Leo', 'email': 'leo@example.com', 'age': 0,
                    # 'tags': []}

```

**重要细节：**为什么不能写 `tags: list = []`

```

# △ 所有实例共享同一个空列表！
@dataclass
class Bad:
    items: list = []

a = Bad()
b = Bad()
a.items.append("hack")
print(b.items)  # ['hack'] □ 互相污染了

# ✔ 用 default_factory
@dataclass
class Good:
    items: list = field(default_factory=list)

```

## dataclass 进阶选项

```

@dataclass(frozen=True)      # 不可变（类似 namedtuple，但有方法）
@dataclass(order=True)      # 自动生成 __lt__、__gt__ 等比较方法
@dataclass(kw_only=True)    # 必须关键字传参（3.10+）
@dataclass(slots=True)      # __slots__ 优化（3.10+）

```

## dataclass vs namedtuple vs dict

```

from dataclasses import dataclass
from collections import namedtuple
import sys

# dict — 灵活但无结构

```

```

d = {"x": 1, "y": 2}
print(sys.getsizeof(d)) # ~232 bytes

# namedtuple – 不可变、轻量
PointNT = namedtuple("PointNT", ["x", "y"])
p = PointNT(1, 2)
print(sys.getsizeof(p)) # ~64 bytes

# dataclass – 可变的 namedtuple 升级版
@dataclass(slots=True)
class PointDC:
    x: int
    y: int
p = PointDC(1, 2)
print(sys.getsizeof(p)) # ~32 bytes (用了 slots)

```

应用场景 - API 请求/响应模型 - 配置项 - DTO（数据传输对象）- 需要 `__eq__` 的值对象 - 替代 `namedtuple`（需要可变性时）

什么时候不该用 **dataclass** - 需要复杂的 `__init__` 行为（用普通 class + 自定义 `__init__`）- 性能极度敏感（dataclass 构造略慢于手写 class）- 3.7 以下的 Python（不支持）

## 类型注解实战

```

from typing import Optional, Union, Any, Callable, TypeVar
from collections.abc import Sequence, Mapping

# 基本注解
def greet(name: str) -> str:
    return f"Hello {name}"

# Optional / Union (= 3.10+ 可以用 str | None)
def find_user(id: int) -> Optional[dict]:
    ...

# Callable – 函数作为参数的类型
def process(

```

```
    items: list[int],
    callback: Callable[[int], str]
) -> list[str]:
    return [callback(x) for x in items]

# TypeVar – 泛型
T = TypeVar("T")
def first(items: Sequence[T]) -> T | None:
    return items[0] if items else None

# 运行时不会强制检查，但 IDE + mypy 会
```

---

## 今日练习

1. **property** 做校验：实现一个 `EmailField` 类，用 `property` 确保 email 包含 `@`，赋值时自动校验
  2. **ABC** 做插件框架：用 `ABC` + `__init_subclass__` 写一个插件注册器，支持插件按优先级排序执行
  3. **Protocol** 实战：定义 `Comparable` Protocol（实现 `__lt__`），让 `sorted()` 函数类型安全地接受自定义对象
  4. **dataclass** 转 **JSON**：用 `asdict()` 把嵌套 `dataclass` 递归转成 JSON，支持 `list[dataclass]`、`dict[str, dataclass]` 的嵌套场景
- 

明天预告：Day 5 — 并发编程入门（*GIL*、*threading*、*multiprocessing*、*asyncio* 基础、*concurrent.futures*）